Chapter 1: Introduction

After completing this chapter, you will understand what this class is, what topics are covered, and the overall class objectives. You will be introduced to the ModusToolbox™ development ecosystem and will learn how to find documentation online.

Table of contents

Table of contents **1**
1.1 What is this class? 2
1.2 What is the ModusToolbox™ ecosystem? 2
1.2.1 Tools 2
1.2.2 Run-time software assets 3
1.3 Packs 3
1.4 Terminology 3
1.5 Supported families 4
1.6 Supported IDEs and reference flows 5
1.7 Application layers 6
1.7.1 HAL 6
1.7.2 PDL 8
1.7.3 Configurators 8
1.8 Installation 8
1.9 Documentation 8
1.9.1 Tool documentation 9
1.9.2 Asset documentation 9
1.9.3 Developer community 12
1.9.4 Device and solution documentation 13
1.10 Exercises 14
Exercise 1: Download class material 14
Exercise 2: Install the software 15
Exercise 3: Create a developer community account 18
Exercise 4: Look at online documentation 19
Disclaimer **20**

Convention	Usage	Example
`Courier New`	Displays code and text commands	`CY_ISR_PROTO(MyISR);` `make build`
Italics	Displays file names and paths	*sourcefile.hex*
[**bracketed, bold**]	Displays keyboard commands in procedures	[**Enter**] or [**Ctrl**] [**C**]
Menu > Selection	Represents menu paths	**File > New Project > Clone**
Bold	Displays GUI commands, menu paths and selections, and icon names in procedures	Click the **Debugger** icon, and then click **Next**.

1.1 What is this class?

This class is a survey of the ModusToolbox™ development ecosystem. The learning objective is to introduce you to all the tools in the ModusToolbox™ ecosystem and help you develop some familiarity with using them. The class is "a mile wide and an inch deep." This should enable you to understand the scope of the development ecosystem and teach you where to find "everything."

This is a "Level 1" class, meaning that it is intended as an entry point to get you started with the ModusToolbox™ ecosystem. Level 2 classes dig deeper to give more detailed training on specific products such as PSOC™ or XMC™ MCUs. Level 3 classes go even further by diving into a complete solution such as Bluetooth®, Wi-Fi, Motor Control, or Machine Learning.

This version of the training targets ModusToolbox™ tools package 3.8 and uses Microsoft Visual Studio Code (VS Code) throughout. ModusToolbox™ itself does not mandate any particular IDE; Eclipse, IAR Embedded Workbench, and Arm® Keil® µVision® are all fully supported by ModusToolbox™. However, VS Code was chosen for this training because it offers a free, cross-platform, and well-integrated experience for new ModusToolbox™ developers.

1.2 What is the ModusToolbox™ ecosystem?

Before we talk about what the ModusToolbox™ ecosystem is, let's talk for a second about what it *isn't*. It isn't an IDE or a compiler. It isn't just software, and it isn't just a set of libraries or an SDK.

The ModusToolbox™ ecosystem is a collection of software and tools designed to work together and with third-party solutions so you can build the development environment that works best for you. You choose your IDE, your compiler, your RTOS, and your ecosystem without compromising usability or access to ModusToolbox™'s CAPSENSE™, AIROC™ Wi-Fi and Bluetooth®, security, and other features. Out of the box, ModusToolbox™ supports VS Code, Eclipse IDE for ModusToolbox™, IAR Embedded Workbench (Windows only), and Arm® Keil® µVision® (Windows only), as well as a fully scriptable command-line flow.

Note: *This Level 1 Getting Started training uses VS Code throughout. ModusToolbox™ itself is IDE-agnostic — see "Using supported IDEs" in the ModusToolbox tools package user guide for the equivalent steps in Eclipse, IAR, or Keil µVision.*

With the ModusToolbox™ ecosystem, you don't have to choose between closed, proprietary flows that struggle to keep pace with modern innovations and open platforms that fail to support the unique features of your products. The ModusToolbox™ ecosystem provides a "best of both worlds" platform that delivers a great development experience, increased productivity, and feature-rich, robust applications.

The ModusToolbox™ ecosystem is divided into Tools and Run-time Software Assets. Tools are installed as part of the ModusToolbox™ tools install package; Run-time Software Assets are downloaded as needed for an application.

1.2.1 Tools

Tools refer to programs and services that run on the developer's host computer or in the cloud. For example:

- Dashboard
- Compilers (GCC, Arm®, IAR)
- Build System (make, Cygwin)
- Programming and Debug Tools (OpenOCD, PyOCD)
- Configurators and Tuners

Packs

- Project Creator
- Library Manager
- Firmware Loader

Note: *Starting with ModusToolbox™ 3.x, programming and debug tools are a separate ModusToolbox™ Programming tools package installed alongside the tools package via Setup. It includes ModusToolbox™ Programmer, fw-loader, KitProg3 firmware, and OpenOCD. See section 1.2.2.5 of the ModusToolbox™ tools package user guide for more information on Program and debug support.*

1.2.2 Run-time software assets

Run-time Software Assets refers to code that executes on the target device. This includes:

- BSP (Board Support Package)
- Libraries (e.g., RTOS, Network Stacks, Graphics, etc.)
- Customer or Code Example Application Firmware (the project, or one of our code examples)

1.3 Packs

ModusToolbox™ Packs provide a mechanism to add or modify content and tools in a ModusToolbox™ installation. The intent is that ModusToolbox™ Packs deliver content and tools related to a single type of technology provided by Infineon. For instance, there may be Early Access Packs for a new family of devices prior to full public release, or Technology Packs for solutions such as machine learning, USB, or motor control.

Any given pack may include any combination of new content and tools. For example, a Technology Pack for machine learning may include a machine learning configurator, machine learning middleware libraries, and machine learning code examples.

Packs are distributed through the Infineon Developer Center (IDC) using the using the ModusToolbox™ Setup program. Depending on the use case, access to a Pack may require special licensing.

1.4 Terminology

The following are definitions for terms that are used throughout the ModusToolbox™ ecosystem and this class. Some of these terms will be covered in more detail later in this class.

Term	Description
Project	A directory that must contain a Makefile and one or more source files. The Makefile must be built such that it references a board support package (BSP) and can be built using a ModusToolbox™- or BSP-supplied make recipe. The result of the build process is an ELF file.
Application	A directory that contains one or more ModusToolbox™ projects that are meant to build and work together targeting one or more devices in a system. For example, a dual-core application may contain two projects; one for each core.
Workspace	A directory that contains one or more ModusToolbox™ Applications and the common *mtb_shared* ModusToolbox™ asset repository.
Asset	A resource that is referenced by a ModusToolbox™ Project that is retrieved using the ModusToolbox™ software `make getlibs` mechanism, which means it is retrieved using a git clone operation.
BSP	A ModusToolbox™ asset that supports the target device and board. It provides support to boot a typical application such as startup and linker scripts, provides aliases such as pin names to

Supported families

Term	Description
	support cross board common resources, and provides default configurations for some pins, clocks, and peripherals. The customer will typically evolve a standard BSP to support their own board.
Code example	A resource that acts as a template and can support one or more BSPs for a new ModusToolbox™ application. The template is used to initialize the ModusToolbox™ application directory using a `git clone` operation during project creation.
Tool	A host platform (Windows, MacOS, Linux) program that is installed by the ModusToolbox™ tools installer and performs a specific function in the ModusToolbox™ development environment.
Tools package	An installer that provides the set of ModusToolbox™ tools that are required to enable the ModusToolbox™ development environment.
Device Support Library (DSL)	A ModusToolbox™ asset that packages several sub-assets to support a given device, such as the HAL and PDL. These assets are device specific. Legacy devices will not pull a DSL, but instead stand-alone copies of all their dependent assets.
Peripheral Driver Library (PDL)	A ModusToolbox™ asset that contains source code that provides low-level support for the hardware found in a given target device. This library should support all of the features of all of the hardware in a given device. There may be some exceptions based on the business needs for a given device.
Hardware Abstraction Layer (HAL)	A ModusToolbox™ asset that contains source code that is an implementation of a standard API that is common across all devices that support the HAL. The intent is to enable code to be written that is portable to all devices that support the HAL.
Configurator	A ModusToolbox™ tool that provides a graphical interface to ease the process of configuring a target device or middleware asset. Configurators generate data structures that are consumed by the build process.
BSP Configurator	Hardware configurators that interact directly (via PDL or HAL) with the hardware of the device. Files from these configurators are typically provided by the BSP.
Library Configurator	Hardware configurators that are coupled with a firmware library. These configurators generate data structures that are consumed by its associated firmware library. Examples of these include the USB configurator and the Bluetooth® configurator.
Device Configurator	A ModusToolbox™ BSP configurator that provides a graphical interface to configure the basic configuration of the target device including clocks, pins, and various hardware IP blocks. Its information is stored in the file *design.modus*.

1.5 Supported families

The ModusToolbox™ ecosystem supports many Infineon product families, and more are being added all the time. In fact, by the time you read this, there may already be more supported products than are shown here.

Device	Description
PSOC™ Edge MCU	The PSOC™ Edge devices are based on high performance Arm® Cortex®-M55, including Helium™ DSP support paired with Arm® Ethos™-U55 and Cortex®-M33 paired Infineon's ultra-low power NNLite neural network accelerator. Support for "always-on" sensing and response makes the devices ideal for advanced IoT devices such as smart home, security, wearables, robotics, and many more.
PSOC™ Control MCU	PSOC™ Control C3 is a family of high-performance Arm Cortex-M33 microcontrollers for motor control and power conversion applications. This family addresses the need for increased system efficiency, reliability, and flexibility, as well as higher standards for functional safety and security.

Supported IDEs and reference flows

Device	Description
PSOC™ 6 MCU	These are 32-bit dual-core Arm® Cortex®-M4 and Cortex®-M0+ MCUs. They include a rich set of digital and analog peripherals including our industry-leading CAPSENSE™ capacitive-sensing technology. These devices can be paired with Infineon's AIROC™ Wi-Fi, AIROC™ Bluetooth®, or AIROC™ combo radio modules to create secure, low-power, feature-rich IoT products.
PSOC™ 4 MCU	These are 32-bit Arm® Cortex®-M0 and Cortex®-M0+ microcontrollers. As with PSOC™ 6 MCUs, most devices include CAPSENSE™ capacitive-sensing technology as well as many other digital and analog peripherals.
TRAVEO™ MCU	These are 32-bit microcontrollers targeted to industrial and automotive applications.
XMC™ MCU	These are 32-bit industrial microcontrollers including Arm® Cortex®-M0 or Cortex®-M4F CPUs. Some devices include up to 6 cores for large scale industrial systems.
AIROC™ Bluetooth®	This is a family of fully compliant Bluetooth® devices that support basic rate (BR), extended data rate (EDR), and low energy (LE).
AIROC™ Connectivity	This is a family of microcontrollers with integrated Wi-Fi.
PMG1 USB-C Devices	This is a family of high-voltage microcontrollers with USB-C power delivery. The devices include an Arm® Cortex®-M0/M0+ CPU based on the PSOC™ 4 MCU architecture and a USB-C PD controller along with analog and digital peripherals.
Wireless Charging	These are devices used for wireless charging applications.

Note: *Use the Project Creator tool with the default manifest URL to see the current list of supported device families.*

1.6 Supported IDEs and reference flows

As described earlier, the ModusToolbox™ ecosystem isn't just an IDE. Rather, the ecosystem supports multiple IDEs so that you can work the way you want. There are four IDEs that are supported explicitly by the ModusToolbox™ ecosystem, but others can also be used by starting from the command line and adding the necessary IDE support. This training uses VS Code throughout as its example IDE. For more information on installing and using the Infineon ModusToolbox for VS Code extension refer to the VS Code section in exercise 2.

The IDEs that are supported by the ModusToolbox™ ecosystem out of the box are Microsoft Visual Studio Code (VS Code), the Eclipse IDE for ModusToolbox™, IAR Embedded Workbench, and Arm® Keil® µVision®. Each of these can be selected as the target IDE during application creation which will create the files needed for the chosen IDE. Files for IDEs can also be created from the command line for an existing application.

IDE	Description
VS Code	This selection generates json files in the *.vscode* directory containing projects settings, launch configurations, and task configurations to allow easy program and debug operations. It also creates VS Code workspace file called *<app_name>.code-workspace* that allows the application and its associated libraries to be viewed together. For more detailed information see the Visual Studio Code for ModusToolbox™ user guide.
Eclipse IDE for ModusToolbox™	This selection creates Eclipse project files *.project* and *.cproject* and a directory containing launch configurations called *.mtbLaunchConfigs* to allow easy program and debug operations. These files are intended to be used with the customization of Eclipse that is installed by the ModusToolbox™ Setup installer. This customization includes some additional menus and windows, as you will see later. You can use standard Eclipse, but those features will not be available. For more detailed information see the Eclipse for ModusToolbox™ user guide.

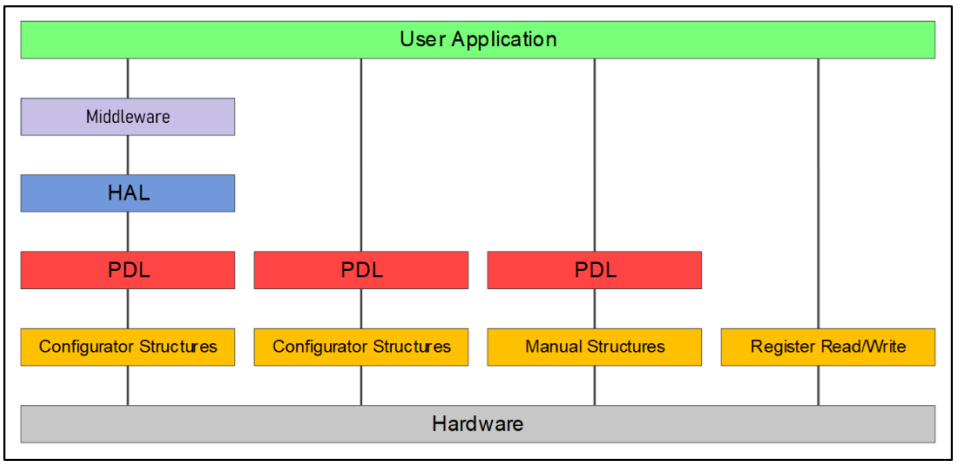
Application layers

IDE	Description
IAR Embedded Workbench	This selection generates the file *<app_name>.ipcf* that is used to open/create the project inside IAR Embedded workbench. For more detailed information see the IAR Embedded Workbench for ModusToolbox™.
Arm® Keil® µVision®	This selection generates the file *<app_name>.cprj* necessary for µVision. For more detailed information see the Keil µVision for ModusToolbox™ user guide.

1.7 Application layers

Many devices supported by the ModusToolbox™ ecosystem provide methods with different levels of abstraction to interact with the hardware. These range from direct register read/write with complete control to higher level abstraction layers that simplify the interface and provide portability between devices and families.

The description below is specifically for PSOC™ Edge MCUs, but other product families use the same or similar concepts. There are four distinct ways for a PSOC™ Edge MCU application to interact with the hardware as shown in the following diagram:



- **Configurator Structures**: Application code uses PDL via structures created by a configurator.
- **Manual Structures**: Application code uses PDL through structures created manually.
- **Register Read/Write**: Application code uses direct register read and writes.

Note: A single application may use different methods for different peripherals.

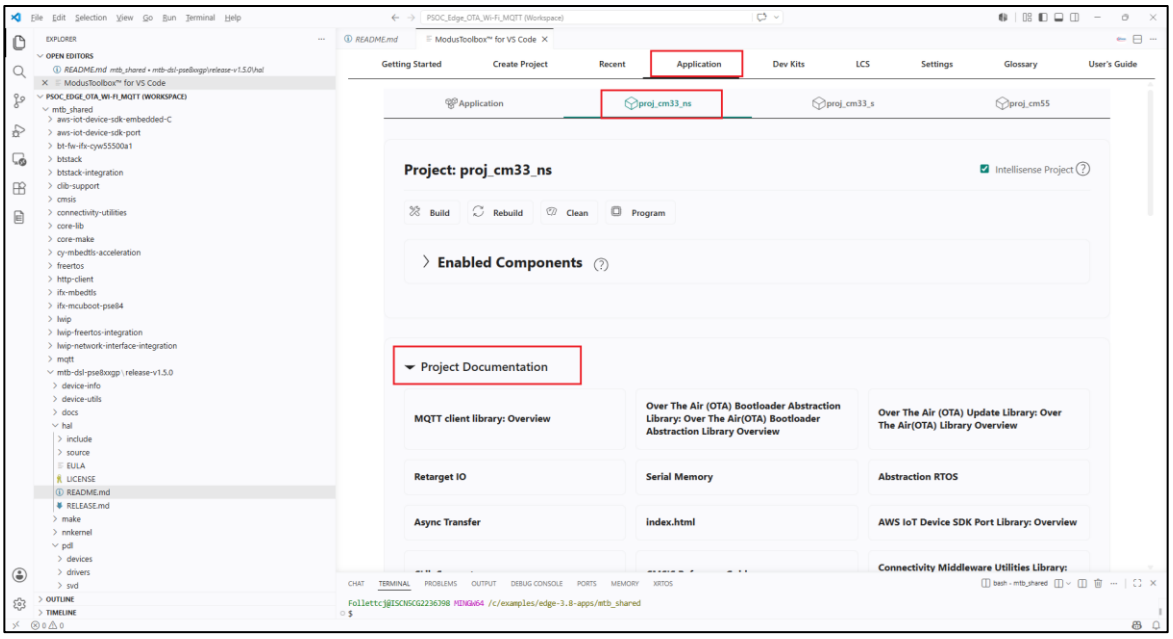
1.7.1 HAL

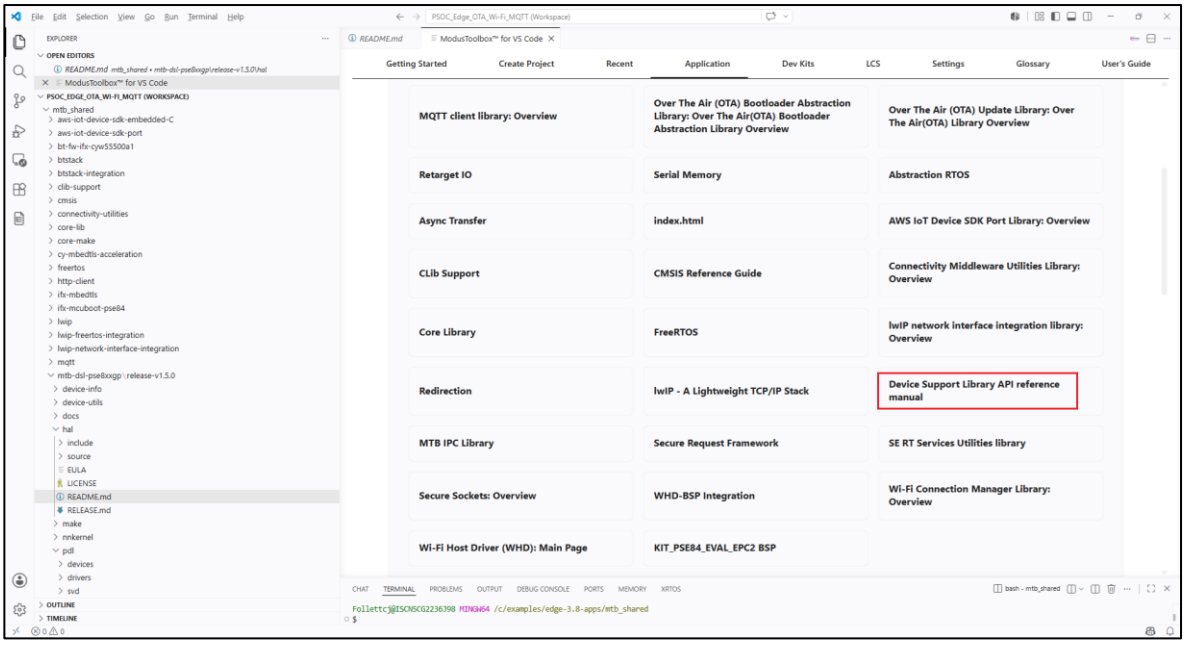
The primary intended use for the HAL is Infineon middleware. It is not intended to be a full feature abstraction layer, but provide just enough support to enable middleware to be as portable as possible within ModusToolbox™. It is accessible to the user application and can be used if the limited functionality satisfies requirements and portability is desired. The HAL is packaged within the DSL.

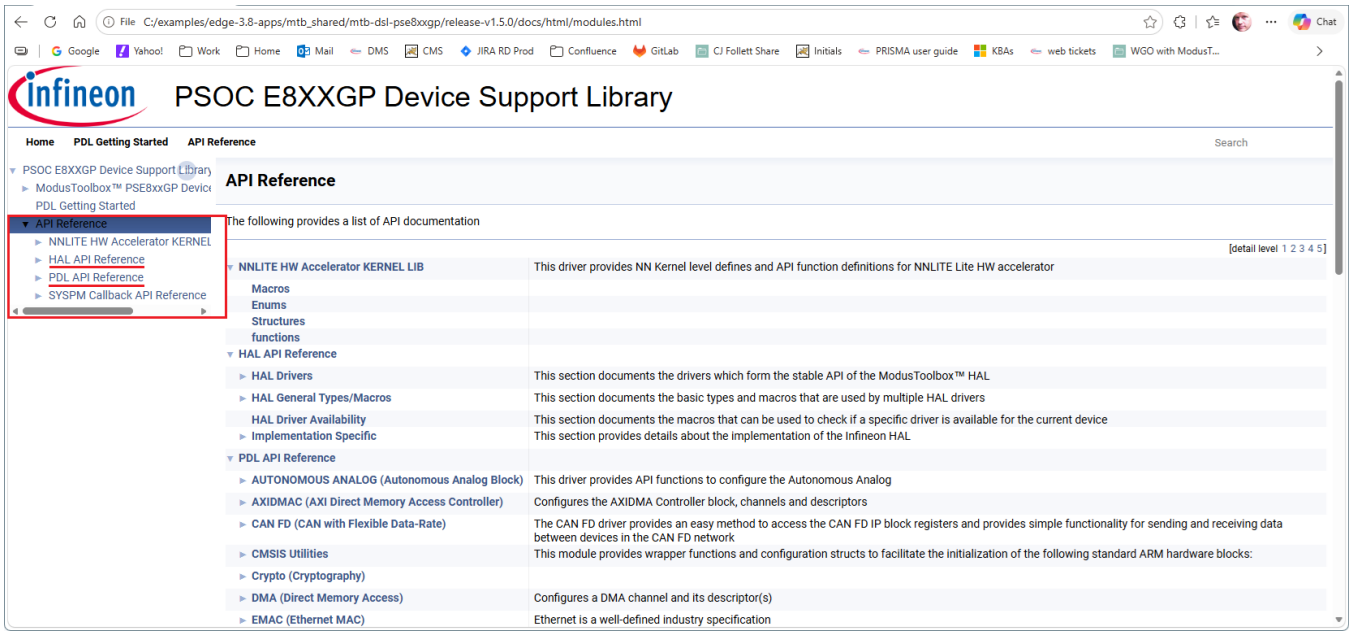
Legacy versions of the HAL, indicated by an asset naming scheme of mtb-hal-cat, were designed to be a user-friendly alternative to the PDL with a focus on portability. It was the preferred method for simpler functions and applications without strict flash size limitations.

After creating a PSOC™ Edge MCU project, open the project in VS Code with the Infineon ModusToolbox™ for VS Code extension. Click the **ModusToolbox™ for VS Code** icon in the Activity Bar to open the sidebar.
Under Project **Documentation**, click any asset link to open its API reference in your default browser.

Application layers







1.7.2 PDL

The PDL is a lower-level interface to the hardware (still simpler than direct register access) that supports all hardware features. The PDL usually goes together with configurators. Because the PDL interacts with the hardware at a lower level it is less portable between devices, especially those with different architectures.

After creating a PSOC™ Edge MCU project, open the project in VS Code with the extension. Click the **ModusToolbox for VS Code** icon in the Activity Bar to open the sidebar. Under **Project Documentation**, click the PDL link to open the API reference in your default browser.

1.7.3 Configurators

Configurators make initial setup easier for hardware accessed using the PDL. The configurators create the necessary structures based on your selections, without you needing to know the exact composition of each structure. Configurators are discussed in more detail in later chapters.

1.8 Installation

As you learned earlier, the ModusToolbox™ ecosystem consists of tools and software assets. The tools are installed onto your computer using the ModusToolbox™ Setup program while software assets are downloaded as needed from GitHub. For more information refer to the ModusToolbox™ software installation guide.

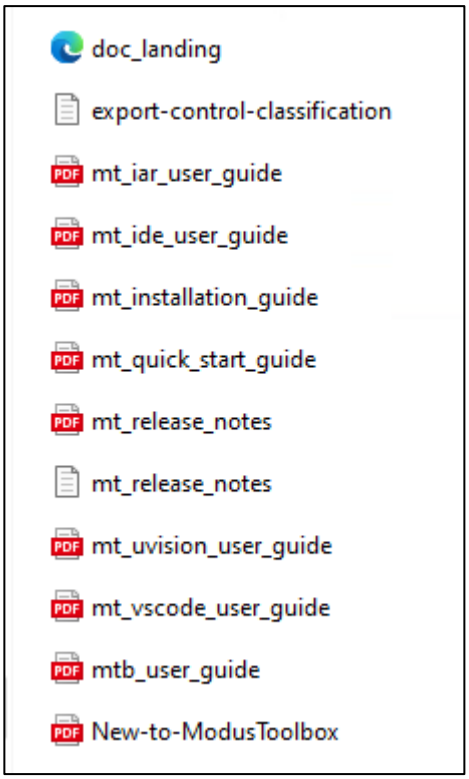
1.9 Documentation

There is considerable documentation for the ModusToolbox™ ecosystem. This includes tool documentation, asset documentation, code example documentation, and developer community posts. In addition, our devices and solutions each have their own documentation. All of it is available online; some is also available offline. A summary of some of the existing documentation is provided below.

1.9.1 Tool documentation

The tool documentation is provided as part of the installation. Top level documentation can be found in *<ModusToolbox™ Installation Directory>/ModusToolbox/docs_<version>*. The installation directory defaults to the user's home directory.

The documentation includes an HTML file that has links to documentation available on disk for all the tools as well as links to files on GitHub and other Infineon pages. The documentation directory also has the installation guide, release notes, user guide, and the user guide and quick start guide for the customized Eclipse IDE for ModusToolbox™. For example:



From the web, you can go to https://www.infineon.com/cms/en/design-support/tools/sdk/modustoolbox-software/ and look at the **Documentation** section.

You can also find documentation for a specific tool inside the docs directory of the tool itself. The path is *<ModusToolbox™ Installation Directory>/ModusToolbox/tools_<version>/<toolname>/docs.*

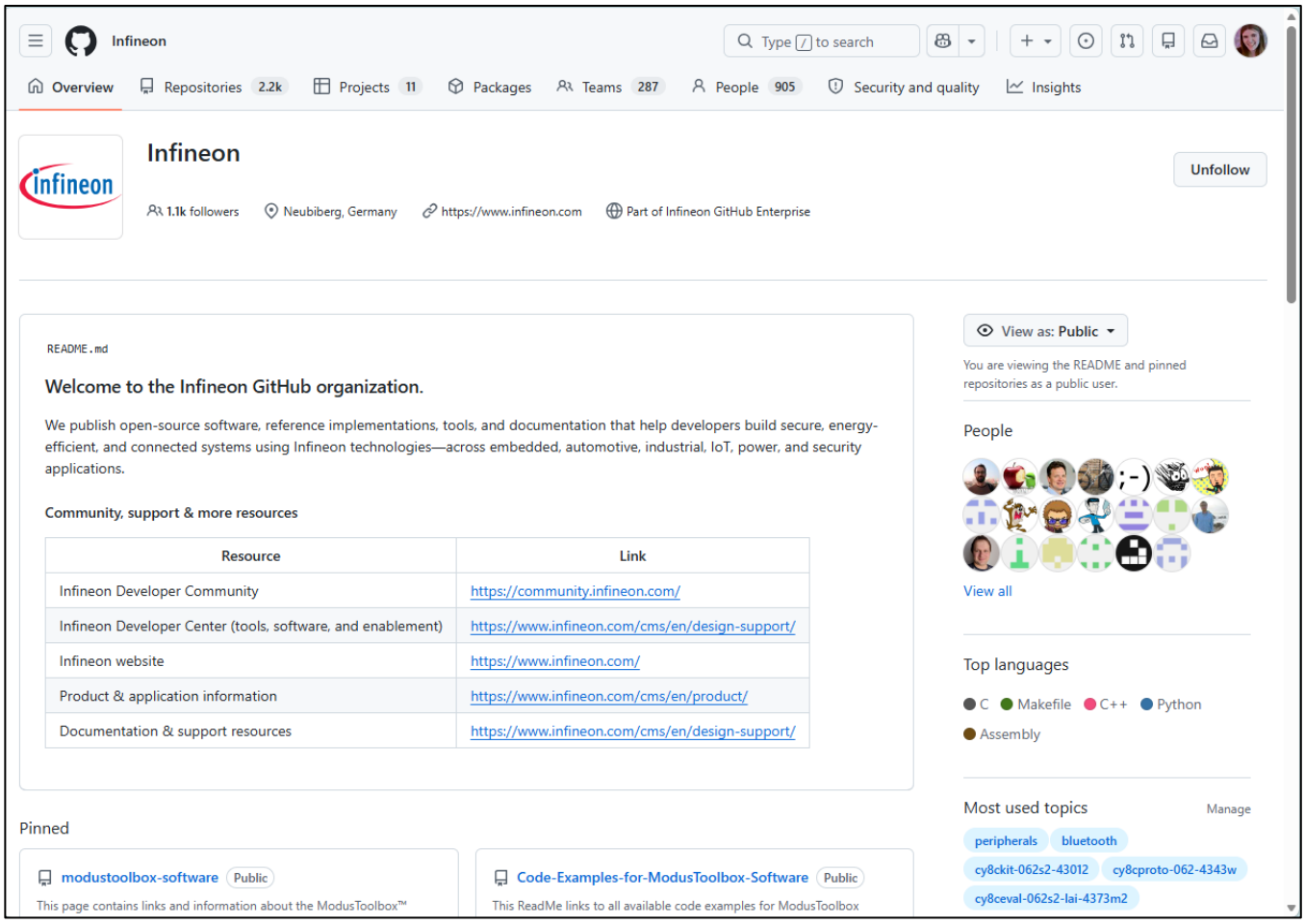
1.9.2 Asset documentation

Assets such as BSPs, libraries, and code examples each have their own documentation. Each asset has a README.md file that can be viewed directly on GitHub. It can also be viewed locally once the asset is downloaded — VS Code includes a built-in Markdown preview (right-click the file > Open Preview, or press [Ctrl] [Shift] [V]), and the ModusToolbox™ for VS Code sidebar provides one-click access to the README of every asset associated with the currently open application.

The main GitHub site is: https://github.com/Infineon

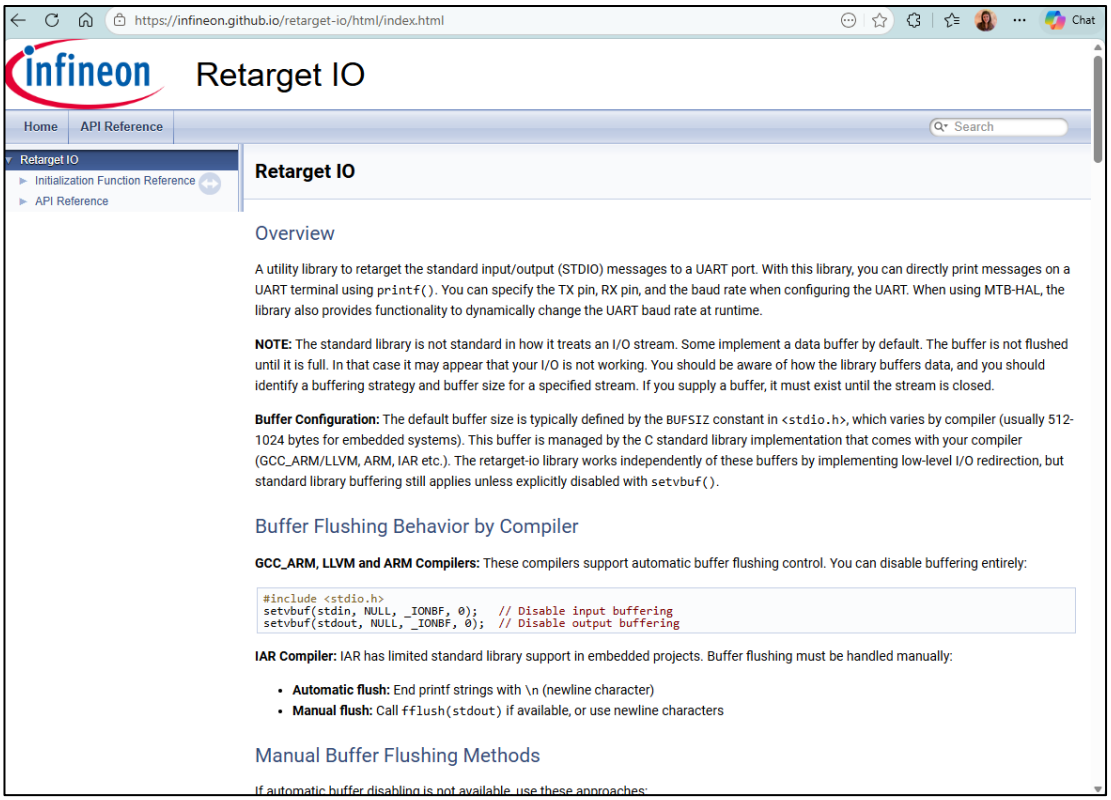
From there, you can search for repos by name or you can select one of the links to filter out code examples, or other software assets.

Documentation



Most assets also have a directory named docs containing an HTML file with the API reference and other general information. For example, the retarget-io library has the following HTML file:

Documentation

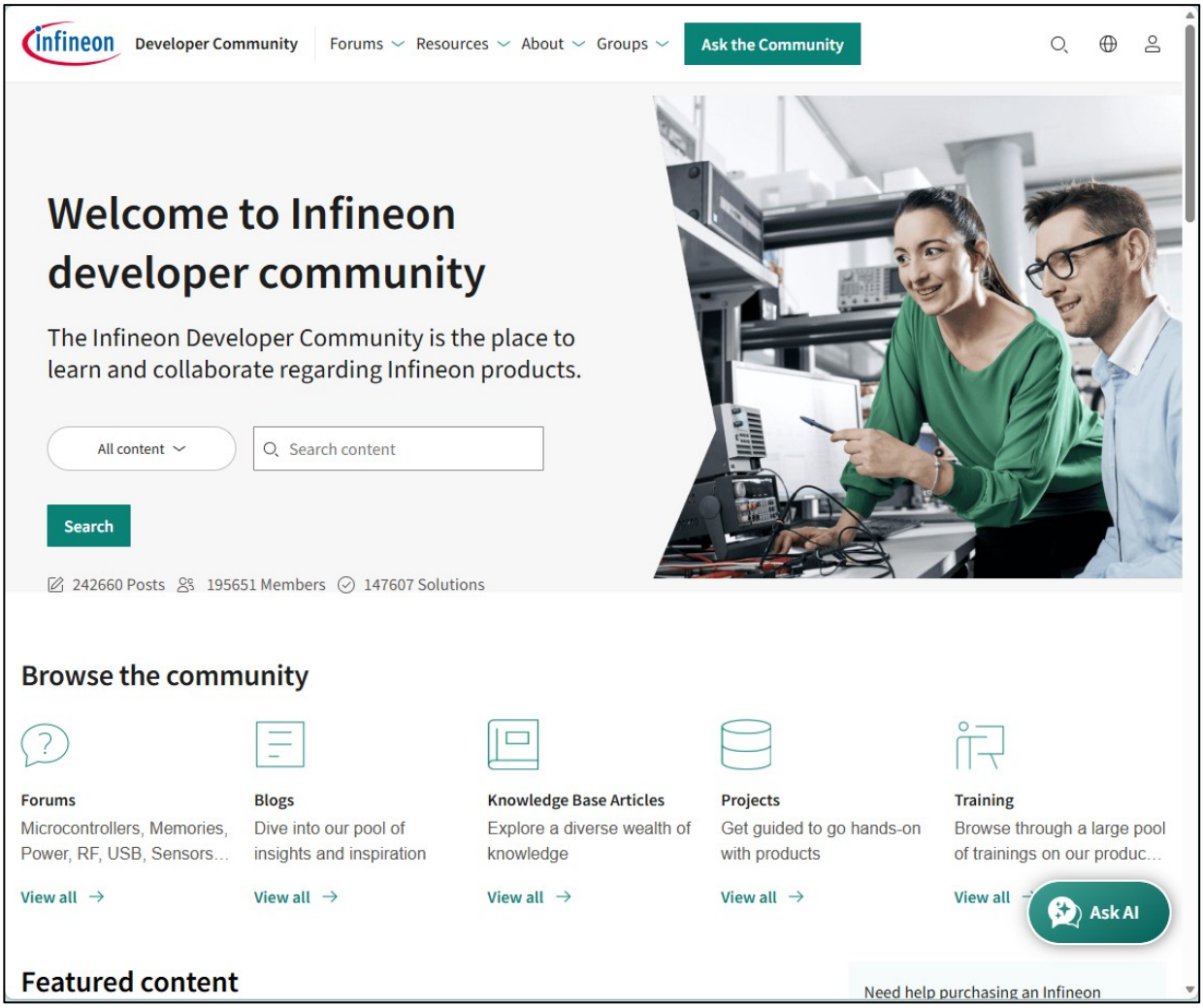


Note: *To view an asset's HTML documentation directly from GitHub Pages, use the URL pattern https://infineon.github.io/<asset_name>/html/index.html (assuming the documentation is in docs/html) and replace <asset_name> with the name of the asset.*

The HTML files associated with the assets that are part of an application are listed in the ModusToolbox™ for VS Code sidebar in VS Code whenever that application is the active workspace. Click any entry to launch the HTML file in your default browser directly from inside VS Code.

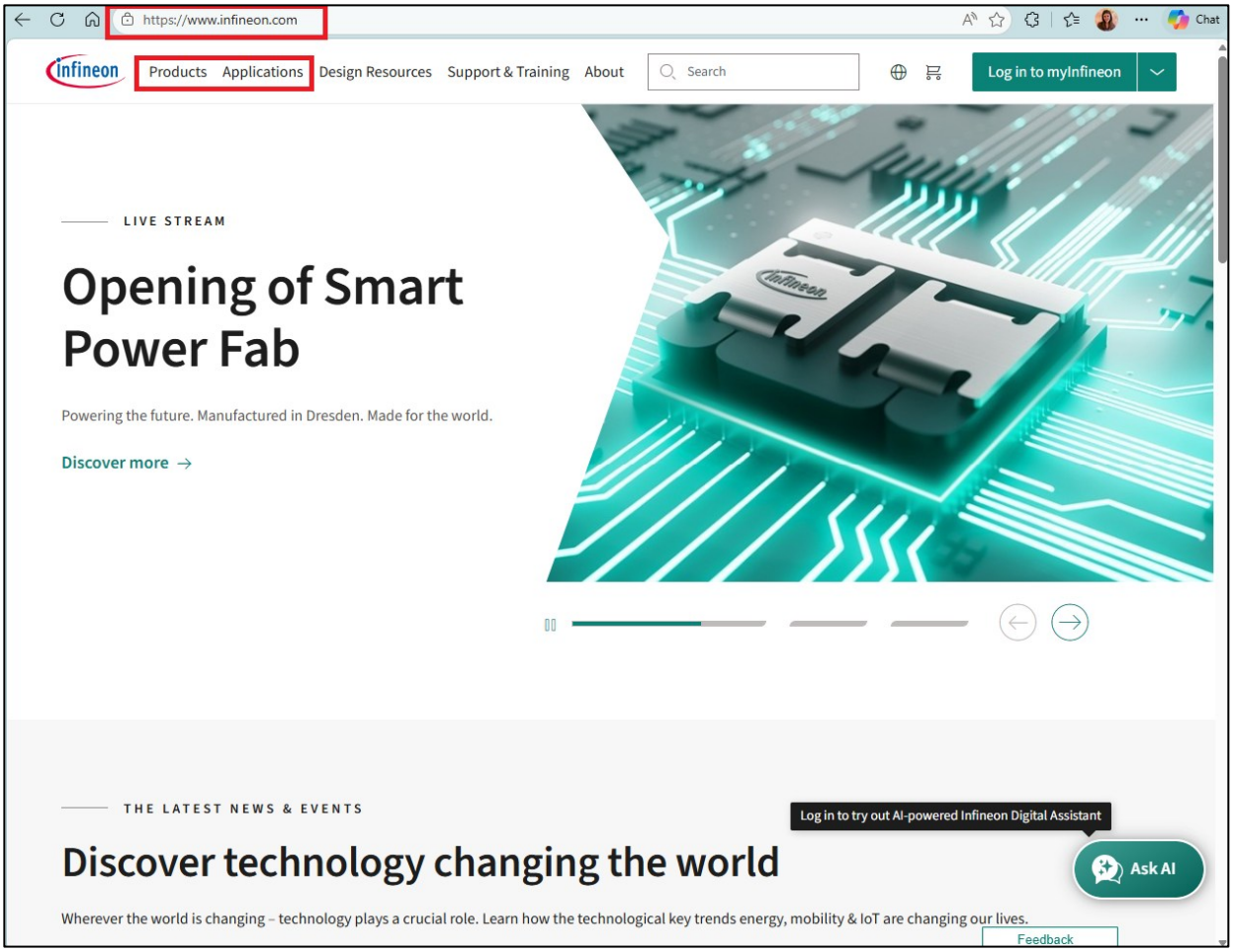
1.9.3 Developer community

There is an active community that can be found at https://community.infineon.com. Once you have registered for a free account, you can review existing posts and start your own discussions. Infineon has Application Engineers following the community that are available to answer your questions and troubleshoot potential issues.



1.9.4 Device and solution documentation

Device datasheets and solution documentation can be found on the web. Start at https://www.infineon.com and use either the **Products** or **Applications** tabs to find what you are looking for.

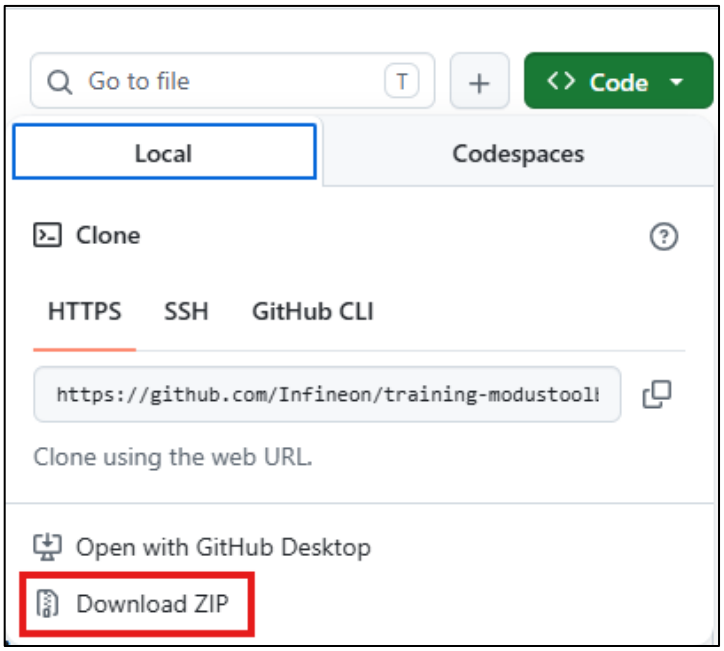


1.10 Exercises

Exercise 1: Download class material

In this exercise, you will download the class material from GitHub. This gives you local access to the manuals and projects.

1. Use a Web browser to go to the class GitHub site at: https://github.com/Infineon/training-modustoolbox-level1-getting-started
2. Click the **Code** button.



3. Click the **Download ZIP** button to download the repo to your local disk to a convenient location. Then unzip it.

Note: *If you are familiar with Git operations, instead of downloading a ZIP file you can choose to clone the repository to your local disk using the URL.*

Exercises

Exercise 2: Install the software

In this exercise, you will install the ModusToolbox™ 3.8 tools package, the Programming tools package, and VS Code.

ModusToolbox™ tools

1. Go to https://www.infineon.com/cms/en/design-support/tools/sdk/modustoolbox-software/
2. Click “Installation Guide” button to open the ModusToolbox™ Installation Guide. You may need to refer to this later.
3. Click the “Download” button. This takes you to the Infineon Developer Center to download the installer for ModusToolbox™ Setup.
4. Follow the instructions in the ModusToolbox™ Installation Guide for your operating system.

Note: *For Windows, administrator rights are required for ModusToolbox™ Setup installation.*

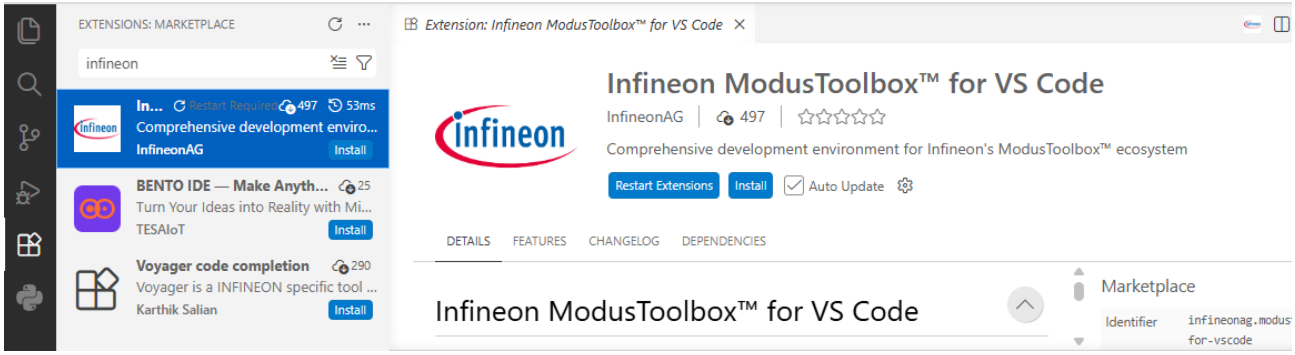
Note: *For Windows, it is best to choose “Install for the current user only.” If you install for all users, you will need to set the CY_TOOLS_PATHS environment variable. See the installation guide section “Installing in non-default location” for further details.*

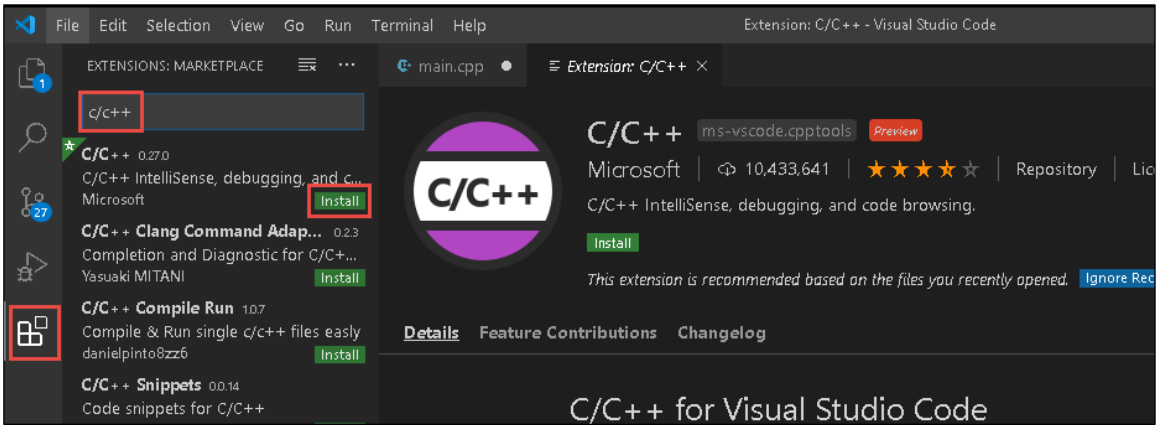
One of the key components of ModusToolbox™ is the make system. It does not support spaces in file or pathnames. Therefore, if your home directory contains spaces you must install in a different location such as C:\ModusToolbox. In that case, once the installation is done, you must create two additional directories and to set four environment variables as described in the ModusToolbox™ Installation Guide section titled "Installing with spaces in user home directory."

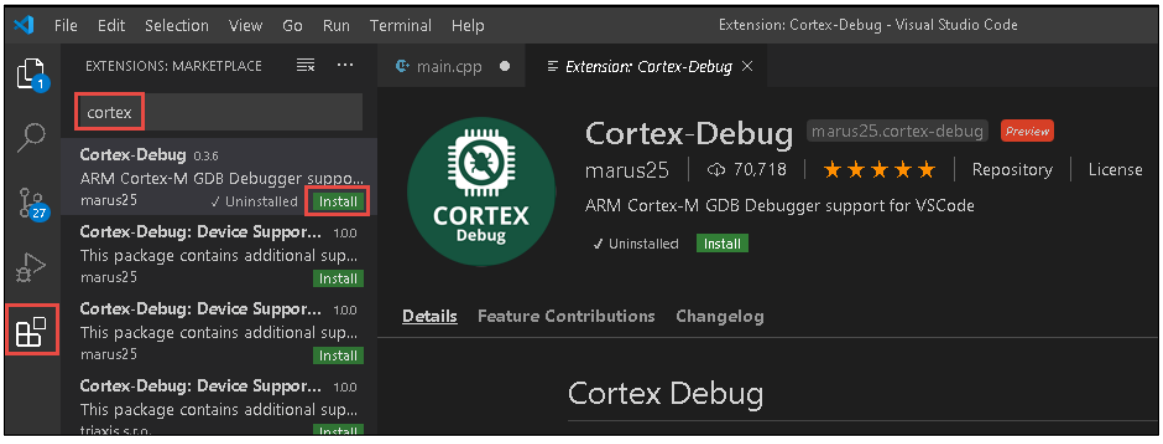
VS Code

1. Download the software from this website:
   https://code.visualstudio.com
2. Follow the instructions for your operating system at:
   https://code.visualstudio.com/docs/setup/setup-overview
3. From the GUI, click the **Extensions** button along the left side, then search for Infineon ModusToolbox™ for VS Code extension, `C/C++` and `Cortex` one at a time. For example:

Exercises





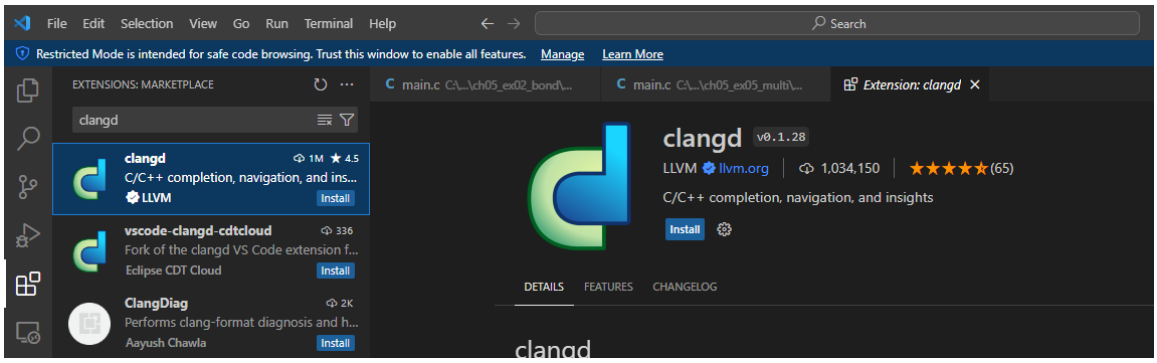


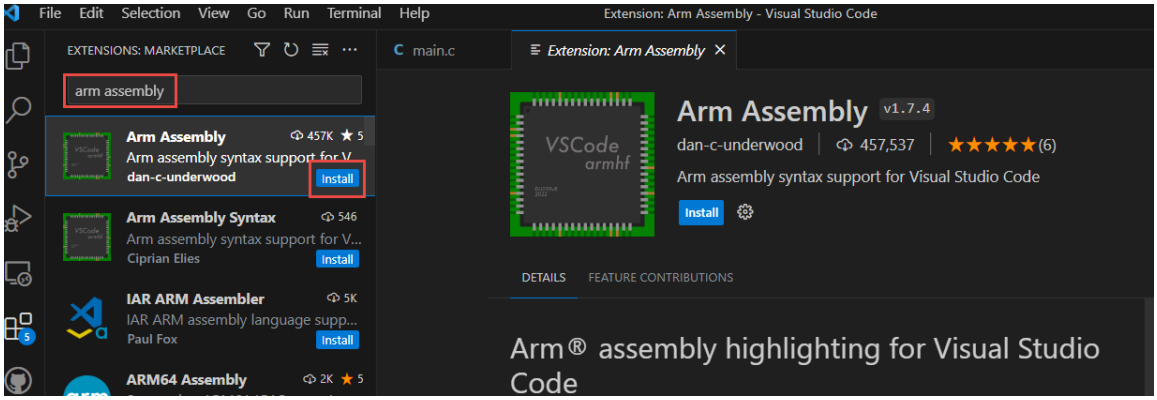
☐ 4. Install, `clangd` and `Arm Assembly` extensions (these are recommended, but not required).

Exercises

`Clangd` and `Arm Assembly` extensions are optional but recommended to improve the development, intellisense and debug experience.

`Clangd` and `Arm Assembly` extensions are optional but recommended to improve the development, intellisense and debug experience.





Note: *These extensions are not Infineon software.*

Windows Subsystem for Linux (WSL)(optional)

If you are using a Windows machine, some operations, such as builds, take significantly longer than they do on Linux. If you want to experience better performance on Windows, you can use the Linux version of ModusToolbox™ with WSL.

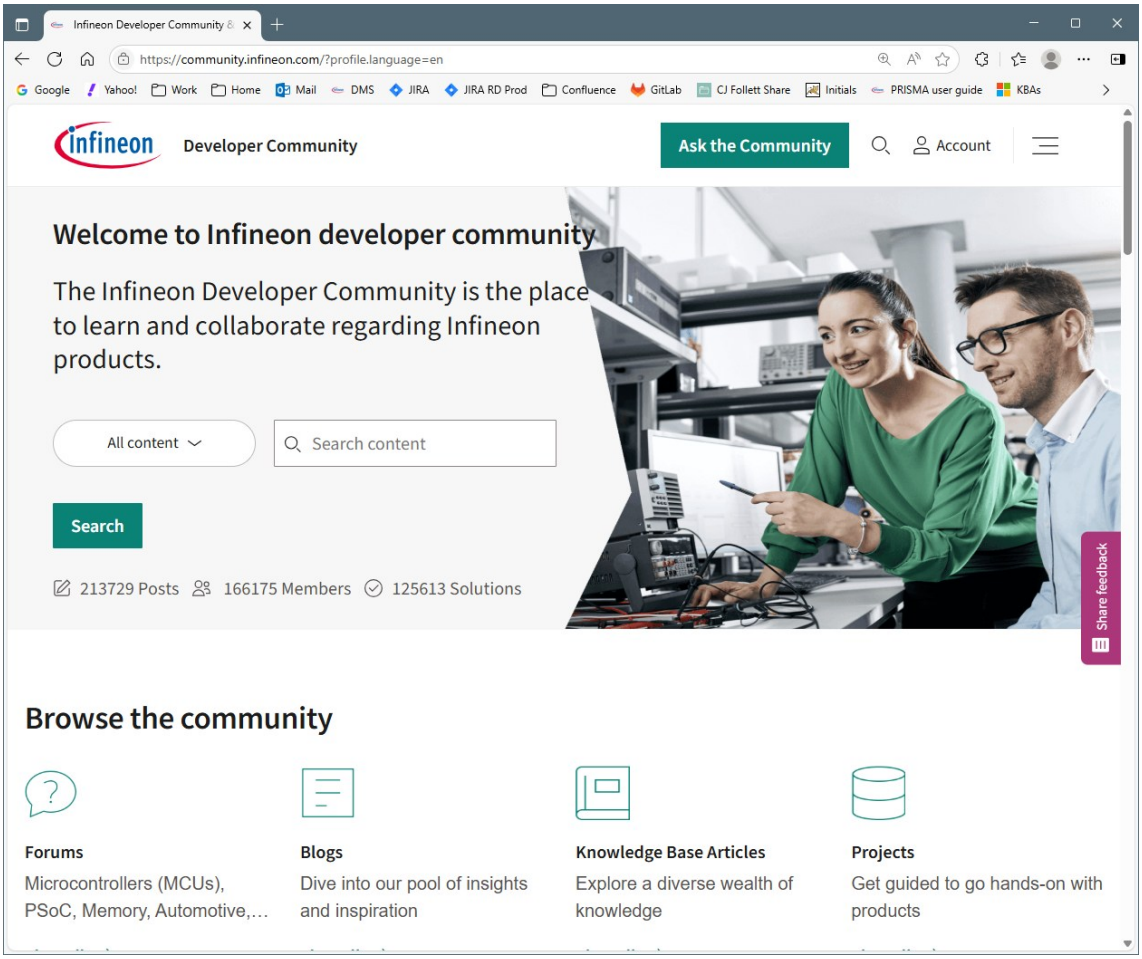
1. Download Application Note AN238846 and follow the instructions to set up and use ModusToolbox™ in WSL.

Exercise 3: Create a developer community account

The developer community is a very useful resource to get answers to just about any question related to the ModusToolbox™ ecosystem or Infineon devices. If you don't already have an account, you should create one now.

1. Open a web browser and go to https://community.infineon.com.
2. Click the **Sign In** button in the upper right corner.
3. If you have an account already, sign in. If not, register for a free account.
4. Once you are logged in, explore the discussions to find topics that interest you.

You can use the bar along the left side to help narrow your search down to specific products, software tools, general topics such as blogs and knowledge base articles, and even code examples contributed by members.



Exercise 4: Look at online documentation

In this exercise, you will become familiar with how to find online tool, asset, code example, and product documentation.

1. Open a web browser and go to https://www.infineon.com/cms/en/design-support/tools/sdk/modustoolbox-software/.
2. Go to the **Documents section**.
3. Explore the list of documents.

Note: *Make sure you are logged into your Infineon account so that you can access all available documentation.*

4. Go to https://github.com/Infineon.
5. Click on the **modustoolbox-software** link.
6. Explore the various run-time software assets and look at a few of the *README.md* files.
7. Go to https://github.com/Infineon again.
8. Click on the **Code-Examples-for-ModusToolbox-Software** link followed by a link of your choice such as PSOC™ MCU, XMC™, PMG1, etc.
9. Explore the list of code examples and look at a few of the *README.md* files.

Trademarks
All referenced product or service names and trademarks are the property of their respective owners.

Edition: 2026

Published by
Infineon Technologies AG
Am Campeon 1-15
85579 Neubiberg, Germany

© 2026 Infineon Technologies AG.
All rights reserved

Do you have a question about this document?
Email: erratum@infineon.com

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

If this document is marked as 'Preliminary', 'Target', 'Draft' or 'Proposal', Infineon reserves the right to change all information given in this document at any time without notice. Subject to the development and release of a Product for series supply by Infineon, the technical specifications of the Product are set forth in the relevant datasheet provided by Infineon without such marking.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).

If the Product includes security features:
Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:
The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved.

Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited.

For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com.